ORACLE SQL & PL/SQL

Industry-Level & Job-Ready Learning Roadmap

Goal: Build practical database skills for entry-level Oracle SQL / PL/SQL developer roles, and demonstrate them through portfolio projects and interview practice.

Duration: 16 weeks **Suggested effort:** 2–3 focused hours/day, 6 days/week.
Target roles: Junior Oracle SQL Developer, SQL Developer Trainee, PL/SQL Developer Trainee, SQL-focused Application Support.

How to use this roadmap

- Follow the phases in order. Become comfortable with SQL before moving deeply into PL/SQL.
- Type every query yourself in Oracle SQL Developer or another Oracle-compatible environment.
- Practise with realistic tables and business data, not only isolated syntax examples.
- Keep a notebook of each problem, query, output, mistake, and correction.
- A topic is complete when you can explain it, write it without copying, and adapt it to a changed requirement.

Scope

Oracle Database is the platform; SQL queries and changes data; PL/SQL is Oracle's procedural extension for blocks, procedures, functions, packages, and triggers. This is a developer-focused roadmap, not a full Database Administrator (DBA) roadmap.

1. The 16-week roadmap at a glance

Weeks	Phase	Expected capability
1–2	**SQL foundations**	Create/query tables; filter, sort, insert, update, delete; use data types and constraints.
3–4	**Core SQL**	Aggregates, GROUP BY, HAVING, joins, subqueries, set operators, CASE.
5–6	**Advanced SQL**	CTEs, analytic functions, hierarchical queries, views, sequences, indexes, data dictionary.
7–8	**Data design & transactions**	Normalize schemas; model relationships; understand ACID, commits, rollbacks, and locks.
9–11	**PL/SQL foundations**	Blocks, variables, conditions, loops, cursors, exceptions, procedures, functions.
12–13	**Professional PL/SQL**	Packages, triggers, collections, bulk processing, dynamic SQL, maintainable code.
14	**Performance & testing**	Basic execution plans, indexing trade-offs, debugging, test cases, error diagnosis.
15–16	**Projects & job prep**	Complete portfolio projects, practise interviews, explain design choices, prepare resume.

2. Detailed syllabus by phase

Weeks 1–2: SQL foundations

- Relational database concepts: schemas, tables, rows, columns, primary keys, and foreign keys.
- Oracle data types: NUMBER, VARCHAR2, CHAR, DATE, TIMESTAMP, and CLOB; choose appropriate types.
- DDL: CREATE, ALTER, DROP, TRUNCATE. Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK.
- DML: INSERT, UPDATE, DELETE. Transactions: COMMIT and ROLLBACK.
- SELECT, DISTINCT, aliases, WHERE, comparison operators, AND/OR/NOT, IN, BETWEEN, LIKE, IS NULL.
- ORDER BY, expressions, concatenation, arithmetic, and NULL behaviour.

Weeks 3–4: Core and intermediate SQL

- Single-row functions: character, numeric, date, conversion, and general functions such as NVL and COALESCE.
- Aggregate functions: COUNT, SUM, AVG, MIN, MAX; GROUP BY and HAVING.
- Joins: INNER, LEFT, RIGHT, FULL OUTER, CROSS, self joins, and multi-table joins.
- Subqueries: single-row, multiple-row, correlated; EXISTS and NOT EXISTS.
- Set operators: UNION, UNION ALL, INTERSECT, MINUS.
- CASE expressions, conditional aggregation, top-N and pagination with FETCH FIRST / OFFSET.

Weeks 5–6: Advanced SQL

- Common table expressions (WITH clause), including multi-step query design.
- Analytic functions: ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM/AVG OVER, PARTITION BY.
- Window functions for running totals, top-N per group, latest row per customer, and duplicate detection.
- Views and materialized-view concepts; sequences and synonyms.
- Indexes: why they help, common concepts, trade-offs, and when they may not help.
- Data dictionary: USER_TABLES, USER_TAB_COLUMNS, USER_CONSTRAINTS, USER_INDEXES, USER_OBJECTS; compare USER_, ALL_, and DBA_ views.
- Hierarchical queries: START WITH, CONNECT BY, LEVEL.

Weeks 7–8: Data modelling and transactions

- ER diagrams; entities, attributes, relationships, cardinality, and optionality.
- Primary, candidate, alternate, composite, and foreign keys.
- Normalization: 1NF, 2NF, 3NF; identify insert, update, and delete anomalies.

- Design tables for a workflow; choose data types and constraints deliberately.
- ACID properties; transaction boundaries; COMMIT, ROLLBACK, SAVEPOINT.
- Read consistency, locks, blocking, and deadlocks at a conceptual level.

Weeks 9–11: PL/SQL foundations

- Block structure: DECLARE, BEGIN, EXCEPTION, END; anonymous blocks.
- Variables, constants, %TYPE, %ROWTYPE, assignment, operators, and scope.
- IF / ELSIF / ELSE; CASE; basic and nested loops; EXIT and CONTINUE.
- SELECT INTO; handle NO_DATA_FOUND and TOO_MANY_ROWS.
- Implicit and explicit cursors; cursor FOR loops; cursor attributes.
- Predefined and user-defined exceptions; RAISE and RAISE_APPLICATION_ERROR.
- Procedures and functions; IN, OUT, IN OUT parameters; return values and invocation.

Weeks 12–13: Professional PL/SQL

- Packages: specification, body, public/private members, and code organization.
- Triggers: BEFORE / AFTER, row-level / statement-level; use cases and risks.
- Collections and records: associative arrays, nested tables, VARRAY concepts.
- BULK COLLECT and FORALL: purpose and basic practical usage.
- Dynamic SQL with EXECUTE IMMEDIATE; bind variables and safe input handling.
- Naming, validation, reusable routines, comments, and meaningful error messages.

Week 14: Performance, testing, troubleshooting

- Use EXPLAIN PLAN and DBMS_XPLAN.DISPLAY to inspect a basic execution plan.
- Understand table scans, index access, joins, cardinality, and why optimizer statistics matter.
- Avoid SELECT * in application queries; retrieve only required columns.
- Prefer set-based SQL when suitable; understand row-by-row PL/SQL overhead.
- Use bind variables; understand parsing and SQL injection risks.
- Test normal cases, empty results, duplicates, NULLs, invalid inputs, and boundaries.
- Practise diagnosing ORA-00942, ORA-00904, ORA-01403, ORA-01422, and ORA-00001.

Weeks 15–16: Projects and job preparation

- Complete two portfolio projects with schema scripts, sample data, SQL reports, and PL/SQL code.
- Write a README: business problem, schema, features, setup instructions, and sample outputs.
- Explain joins, subqueries, analytic functions, indexes, transactions, procedures, packages, and exceptions aloud.
- Solve timed SQL problems; review mistakes and rewrite queries without looking at solutions.
- Prepare concise resume bullets describing what you built and the Oracle skills applied.

3. Portfolio projects that prove practical ability

Build at least two projects end-to-end. A strong project includes schema design, constraints, realistic data, useful reports, PL/SQL logic, test cases, and documentation—not just standalone SELECT statements.

Project A: Retail / E-commerce Order Management

- Tables: CUSTOMERS, PRODUCTS, ORDERS, ORDER_ITEMS, PAYMENTS, INVENTORY.
- SQL reports: customer order history, monthly revenue, best-selling products, customers with no orders, low-stock products, top products by category.
- Advanced SQL: monthly running revenue, rank products by sales, latest order per customer, repeat-customer analysis.
- PL/SQL: create-order procedure, order-total function, order-management package, insufficient-stock exception handling.
- Use primary/foreign keys, unique and check constraints, and rollback when an order cannot be completed.

Project B: Student / Training Institute Management

- Tables: STUDENTS, COURSES, BATCHES, ENROLLMENTS, ATTENDANCE, PAYMENTS, EXAMS, MARKS.
- SQL reports: attendance percentage, student performance, fee dues, course-wise enrolment, missed exams.
- Advanced SQL: rank students per batch, identify top performers, calculate running averages.
- PL/SQL: record-marks procedure, grade function, student-report package, justified audit trigger.

Project C (optional): Employee & Payroll

- Tables: EMPLOYEES, DEPARTMENTS, JOBS, SALARY_HISTORY, ATTENDANCE, PAYROLL.
- SQL reports: department salary summaries, employees above department average, salary changes, headcount trends.
- PL/SQL: payroll calculation procedure, validation function, exception handling, controlled audit logging.

Publish these files for each project

- 01_schema.sql — tables, constraints, sequences, and indexes.
- 02_sample_data.sql — realistic sample rows; do not use real personal information.
- 03_reports.sql — 15–25 useful queries from basic to advanced.
- 04_plsql.sql — procedures, functions, packages, and justified triggers.
- README.md — setup steps, ER diagram, features, sample outputs, and lessons learned.

4. Weekly study routine

Time	Activity
30 min	Revise yesterday's topic and explain it in your own words.
45 min	Learn one new concept; note syntax and examples.
60–75 min	Hands-on practice: 5–10 queries or 1–2 PL/SQL exercises.
15 min	Record errors, fix them, and explain why the correction works.

Weekly checkpoint: Solve a small set of questions without notes, explain your approach aloud, and revisit weak topics. Keep one day lighter for revision or rest.

5. Interview preparation checklist

SQL topics to explain and demonstrate

- DELETE vs TRUNCATE vs DROP.
- Primary key vs unique key; primary key vs foreign key.
- WHERE vs HAVING; GROUP BY rules; handling NULL.
- INNER JOIN vs LEFT JOIN; self join; joining multiple tables.
- Subquery vs join; EXISTS vs IN, including NULL considerations.
- UNION vs UNION ALL; INTERSECT and MINUS.
- ROW_NUMBER vs RANK vs DENSE_RANK; PARTITION BY and analytic ORDER BY.
- Find duplicates, second-highest salary, top-N per department, and latest record per customer.
- What an index does, when it helps, and why too many indexes can hurt writes.
- COMMIT vs ROLLBACK vs SAVEPOINT; basic ACID properties.

PL/SQL topics to explain and demonstrate

- SQL vs PL/SQL; anonymous block structure.
- Procedure vs function; IN, OUT, and IN OUT parameters.
- Implicit vs explicit cursor; cursor attributes.
- Predefined vs user-defined exceptions; NO_DATA_FOUND and TOO_MANY_ROWS.
- Package specification vs package body.
- BEFORE vs AFTER trigger; row-level vs statement-level trigger.
- BULK COLLECT and FORALL at a basic practical level.
- How you test and debug a procedure or function.

Practical interview drill

- Solve two SQL problems daily: one query-writing problem and one output/reasoning problem.
- Explain each query line by line, including why each join and filter is needed.
- Practise with unfamiliar table names and changed requirements.
- For each project, explain its business problem, schema, three queries, one PL/SQL routine, and one bug you fixed.

6. Tools and learning resources

- Oracle SQL Developer: create connections, run scripts, inspect objects, and debug SQL/PLSQL.
- Oracle Database Free or another authorized practice database: practise locally or in a permitted hosted environment.
- Official Oracle SQL Language Reference and PL/SQL Language Reference: verify syntax and behaviour.
- GitHub: version-control scripts and projects; include a clear README and sample outputs.
- Practise on HR-style employee/department data plus your own retail or student schema.

7. Job-ready self-assessment

Start applying for junior SQL / PL/SQL roles when you can do most of these without step-by-step help:

- Design a small relational schema with appropriate primary and foreign keys.
- Write joins across 3–5 tables and solve grouping, subquery, and analytic-function problems.
- Explain NULL handling, transaction behaviour, and basic indexing trade-offs.
- Create and debug a procedure, function, package, and a sensible trigger.

- Handle exceptions and validate input instead of allowing unexplained failures.
- Complete two documented projects and explain the business problem each solves.
- Solve common SQL interview questions under time limits and communicate your reasoning.

Final guidance

Do not memorize hundreds of commands. Employers value the ability to understand a requirement, design appropriate tables, write correct and readable SQL, handle edge cases, and troubleshoot. Build depth in SQL first, then PL/SQL, and use projects to prove you can apply both.

Scope note: This plan is developer-focused. Production DBA work—backup and recovery, installation, patching, high availability, capacity planning, and deep database administration—requires a separate DBA roadmap.